

INFORME MAESTRO — GOBERNANZA, ARQUITECTURA Y POLÍTICAS OFICIALES DEL MOTOR IDUNEX

0. Propósito

Este documento define la fuente maestra de gobernanza para el motor IDUNEX, sus proyectos derivados, sus actualizaciones, sus auditorías, sus investigaciones, sus contratos de entrega y su uso en agentes.

Debe servir para cualquier versión futura del motor, no solo para una versión específica. Por eso este documento no fija `v1.0.0` como regla permanente. La versión concreta será definida por el prompt de corrección, generación o actualización que se ejecute en cada caso.

Formato genérico de versión:

```
SEMANTIC_VERSION=x.x.x
```

Ejemplos permitidos según corresponda:

```
v1.0.0  
v1.1.0  
v2.0.0  
vX.Y.Z
```

La permanencia, actualización o salto de versión siempre debe ser dictada por el prompt activo y por la política de release vigente.

1. Principio rector

IDUNEX debe operar como un sistema de autoridad técnica reproducible, auditable y escalable.

El motor no es un proyecto.

El motor no es un historial.

El motor no es un repositorio de logs.

El motor no es una carpeta de experimentos.

El motor no es un archivo de parches acumulados.

El motor es el núcleo de reglas, investigación, validación, generación, actualización, canon, contratos y soporte operativo para crear y mantener proyectos IDUNEX con máxima precisión.

Regla base:

ENGINE_LEVEL ≠ PROJECT_LEVEL ≠ AGENT_LEVEL

Cada nivel tiene responsabilidades propias y no debe contaminar a los demás.

2. Alcance universal del motor

El motor debe poder gobernar:

1. creación de proyectos desde cero;
 2. generación de modelos ficticios adultos;
 3. generación de perfiles 360;
 4. generación de canon visual, voz, personalidad, ropa, ambientes, música, texto y comportamiento;
 5. generación de runtime para agentes;
 6. generación de prompts por modalidad;
 7. uso de investigaciones internas;
 8. validación forense;
 9. actualización de proyectos;
 10. migración de proyectos entre versiones de motor;
 11. actualización de motor;
 12. auditoría de motor;
 13. auditoría de proyecto;
 14. auditoría de carga real en agentes;
 15. control de marca y assets;
 16. control de seguridad;
 17. control de no drift;
 18. control de bloat;
 19. control de duplicados;
 20. control de truthfulness;
 21. control de runtime ChatGPT/Copilot;
 22. control de prompt packs A-J;
 23. control de proyectos con poca, media o completa información;
 24. control de updates por edad evolutiva, marca, modelos, ropa y runtime;
 25. control de Project Demo como validación posterior;
 26. control de cierre oficial y estado productivo.
-

3. Versionado oficial

3.1 Regla general

El motor debe declarar siempre una versión semántica limpia:

SEMANTIC_VERSION=x.x.x

No debe usar sufijos semánticos como parte de la versión oficial.

Prohibido:

```
x.x.x_UNCHANGED
x.x.x_SAFE
x.x.x_FIX
x.x.x_PATCH
x.x.x_HOTFIX
x.x.x_CLEAN
x.x.x_FINAL
x.x.x_REPAIRED
```

Permitido como metadata técnica separada:

```
VERSION_BUMP=YES/NO
CORRECTION_MODE=DIRECT_CANONICAL_NO_PATCH
CORRECTION_SCOPE=<scope técnico>
RELEASE_PHASE=<fase>
```

3.2 Corrección directa

Si el prompt indica mantener la misma versión, debe aplicarse corrección directa:

```
VERSION_BUMP=NO
CORRECTION_MODE=DIRECT_CANONICAL_NO_PATCH
```

Eso significa:

- no fork;
- no patch externo;
- no hotfix;
- no factory paralelo;
- no validator paralelo;
- no sufijo;
- no duplicación de motor;
- no carpeta “versión anterior” activa;
- no salida derivada;
- no motor alternativo;
- no release paralelo.

4. Sistema recomendado de reglas

Las reglas no deben depender indefinidamente de hitos tipo `H382R`, porque eso puede generar una cadena histórica difícil de mantener.

Los hitos pueden existir como trazabilidad temporal, pero la gobernanza activa debe organizarse por familias estables de reglas.

4.1 Familias oficiales sugeridas

GOV – Gobernanza general
VER – Versionado y release
ENG – Motor / engine
PRJ – Proyecto
AGT – Agentes
CAN – Canon / identidad / Profile360
RES – Research corpus / investigaciones
RUN – Runtime 10+N
PMT – Prompt packs
VAL – Validación recomputacional
AUD – Auditoría
UPD – Updates
MIG – Migraciones
BRD – Marca / assets / logo
REF – Referencia visual
SAF – Seguridad / adulto editorial
ZIP – ZIP / companion / cierre
DUP – Duplicados
LIN – Lineage / evidence
BLT – No-bloat / limpieza
CRT – Certificación creativa

4.2 Ejemplo de reemplazo de hitos

En lugar de dejar una regla como:

H382R_EXTERNAL_WHOLE_ZIP_AUTHORITY

Debe pasar a regla estable:

ZIP-EXT-001: Whole ZIP external authority

Y declarar:

Regla histórica de origen: H382R
Estado: integrada al canon activo

4.3 Uso correcto de hitos H

Los hitos H pueden quedar en:

- changelog compacto;
- lineage de corrección;
- evidencia no autoritativa;
- referencia interna histórica mínima.

No deben ser la forma principal permanente de gobernar el motor.

5. Separación oficial de niveles

5.1 ENGINE_LEVEL — Motor

El motor debe contener:

- reglas generales;
- schemas;
- validadores;
- factories;
- contratos;
- políticas;
- investigaciones canónicas;
- diccionarios;
- Profile360;
- TechExt;
- runtime templates;
- prompt pack templates;
- agent load templates;
- update templates;
- migration logic;
- smoke/stress test matrix;
- fixtures mínimos;
- evidence lineage compacta;
- manuales;
- release surfaces;
- prompt canónico de Proyecto Demo;
- suite de plantillas de proyectos.

El motor no debe contener:

- datos reales de una empresa como default activo;
- datos reales de un proyecto específico como default activo;
- nombres de modelos fijos;
- nombres de clientes fijos;
- logos fijos;
- rutas de proyectos;
- hashes de proyectos;

- outputs creativos;
- ZIPs de proyectos generados;
- logs largos;
- staging;
- temporales;
- historial activo;
- versiones viejas activas;
- parches externos;
- forks;
- factories paralelos;
- validators paralelos.

5.2 PROJECT_LEVEL — Proyecto

Un proyecto debe contener datos concretos solo porque fueron entregados como input o generados por el motor para ese proyecto.

Un proyecto puede ser:

- Proyecto Demo oficial de validación;
- proyecto de empresa específica;
- proyecto de cliente;
- proyecto interno;
- proyecto comercial;
- proyecto experimental;
- proyecto de actualización;
- proyecto de migración.

Cada proyecto debe tener su propio:

- owner;
- marca;
- assets;
- modelos;
- canon;
- runtime;
- evidencia;
- manifests;
- auditoría;
- release surfaces.

5.3 AGENT_LEVEL — Agente

El agente trabaja sobre runtime de un proyecto, no sobre todo el motor.

Debe recibir:

10 archivos base + 1 full runtime canon por modelo

No debe depender de memoria externa ni de interpretaciones no documentadas.

6. Política sobre nombres propios, empresas y proyectos

El motor debe ser genérico.

No debe contener como default activo:

- LatinosFlow;
- Proyecto 000 Demo;
- nombres de modelos demo;
- nombres de clientes;
- paletas de marca específicas;
- logos específicos;
- datos de empresa;
- ciudades o campañas fijas;
- rutas de proyectos previos.

6.1 Excepción controlada: Proyecto Demo

El Proyecto Demo puede usar un owner concreto, por ejemplo LatinosFlow, solo cuando se genera como proyecto real de validación posterior.

Eso no convierte esos datos en default del motor.

Regla:

Los datos de Proyecto Demo viven en el proyecto generado, no en el motor activo.

6.2 Excepción controlada: proyecto de marca específica

Si se crea un proyecto para una empresa concreta, esa empresa puede aparecer dentro del proyecto.

Nunca debe volver al motor como default.

7. Proyecto Demo oficial

7.1 Propósito

El Proyecto Demo es la prueba real obligatoria para declarar que el motor quedó productivo.

Debe generarse después del cierre técnico del motor.

No es fixture.
No es plantilla interna.
No es default activo.
No es parte del motor.
Es un proyecto real de validación posterior.

7.2 Rol en el cierre productivo

El motor pasa a estado productivo solo si ocurre la secuencia completa:

1. Motor auditado PASS
2. Proyecto Demo generado PASS
3. Proyecto Demo auditado PASS
4. Runtime Demo cargado en ChatGPT PASS
5. Runtime Demo cargado en Copilot PASS o `VENDOR_LIMITATION_NOT_ENGINE_FAIL` documentado

7.3 Efecto sobre proyectos futuros

Si el Proyecto Demo pasa, eso certifica que el motor base está productivo.

Sin embargo, cada proyecto futuro todavía debe pasar una auditoría propia, porque cada proyecto puede tener:

- datos de marca distintos;
- assets distintos;
- número de modelos distinto;
- modalidades distintas;
- restricciones legales distintas;
- inputs incompletos;
- riesgo de placeholders;
- riesgo de logo no verificado;
- riesgo de drift entre modelos.

Por tanto:

Proyecto Demo PASS = motor productivo
Proyecto futuro = requiere auditoría recomputacional propia, pero no necesariamente matriz máxima completa del motor

7.4 Evitar redundancia

No todo proyecto futuro debe repetir todas las pruebas máximas del motor.

Modelo recomendado:

Nivel	Cuándo aplica	Intensidad
Motor release	Cada cierre de motor	Matriz máxima completa
Proyecto Demo	Antes de declarar motor productivo	Matriz máxima de proyecto + agente
Proyecto normal	Cada nuevo proyecto	Auditoría recomputacional específica
Proyecto crítico	Cliente premium, cambios grandes, muchos modelos	Auditoría extendida
Update de proyecto	Cada modificación	Auditoría delta + no-drift
Migración	Cambio de versión motor	Auditoría de migración + revalidación

8. Investigación canónica del motor

8.1 Rol de las investigaciones

Las investigaciones internas del motor son la base para lograr:

- Profile360;
- TechExt;
- identidad visual;
- física corporal;
- rostro;
- piel;
- cabello;
- edad ficticia adulta;
- guardarropa;
- ambientes;
- poses;
- movimiento;
- cámara;
- iluminación;
- voz;
- música;
- Suno;
- ElevenLabs;
- texto/persona;
- psicología;
- conducta;
- branding;
- seguridad;
- prompt engineering;
- agente readiness;

- vendor handoff.

Estas investigaciones no son bloat si están conectadas, deduplicadas, gobernadas y usadas por el factory.

8.2 Qué debe hacer el motor con las investigaciones

El motor debe:

- almacenar investigaciones normalizadas;
- clasificar investigaciones por dominio;
- conectar investigaciones con schemas;
- usar investigaciones para generar proyectos;
- destilar investigaciones relevantes en cada proyecto;
- incluir evidence pointers;
- preservar trazabilidad;
- evitar copiar bruto innecesario;
- actualizar investigaciones sin romper compatibilidad;
- auditar cobertura de investigación.

8.3 Qué no debe hacer

No debe:

- duplicar investigaciones en múltiples carpetas;
- guardar versiones antiguas de investigaciones como activas;
- copiar todo el corpus bruto en cada proyecto;
- meter investigaciones no relacionadas en runtime de agente;
- generar perfiles imaginados sin respaldo del corpus;
- usar datos obsoletos como actuales;
- mezclar investigación con outputs de prueba.

8.4 Destilación correcta hacia proyectos

Cada proyecto debe recibir:

research-derived canon + evidence pointers + runtime-ready distilled knowledge

No necesariamente todo el corpus bruto.

Regla:

Máxima precisión no significa máxima copia.
Máxima precisión significa cobertura completa, trazable y relevante.

9. Política de proyectos

9.1 Qué debe tener un proyecto

Todo proyecto IDUNEX debe contener:

- PROJECT_MANIFEST;
- PROJECT_BRAND_REGISTRY;
- owner/empresa;
- uso previsto;
- restricciones;
- modelos;
- full Profile360 por modelo;
- TechExt por modelo;
- human-readable canon;
- visual canon;
- voice canon;
- personality canon;
- wardrobe canon;
- environment canon;
- music/persona canon;
- text/persona canon;
- runtime ChatGPT 10+N;
- runtime Copilot 10+N;
- README_FOR_HUMAN_OPERATOR;
- TEST_PROMPTS_BY_MODALITY;
- STATIC_SMOKE_TEST_MATRIX;
- prompt packs A-J;
- manifests;
- evidence lineage;
- companion ledgers;
- duplicate allowlist;
- no-drift ledger;
- release certificate;
- final machine audit summary;
- ZIP final;
- external SHA companion.

9.2 Qué no debe tener un proyecto

Un proyecto no debe contener:

- archivos de motor completos innecesarios;
- factories del motor duplicados;
- validators del motor duplicados salvo runtime/audit necesarios;
- historial de generación;
- ZIPs intermedios;
- staging;
- temp;
- heartbeat;

- logs largos;
- modelos no involucrados;
- datos de otros proyectos;
- assets no autorizados;
- logos inventados;
- PASS declarado sin recomputar.

9.3 Proyecto con poca información

Si el usuario entrega poca información, el motor debe autocompletar de forma segura.

Debe generar:

- valores coherentes;
- modelos ficticios adultos;
- canon completo;
- marca con datos mínimos;
- logo no verificado si no hay asset;
- runtime completo;
- prompts de prueba;
- QA.

No debe inventar datos críticos como:

- logo exacto;
- derechos de marca;
- assets no adjuntos;
- identidad real;
- autorizaciones legales;
- hashes;
- rutas;
- PASS.

9.4 Proyecto con información intermedia

Debe respetar lo entregado y completar lo faltante por política.

9.5 Proyecto con información completa

Debe preservar los datos exactos del input, salvo conflicto con seguridad, legalidad, canon o motor.

10. Política de actualizaciones de proyecto

El motor debe soportar updates sin drift.

Tipos mínimos:

- cambiar edad evolutiva;

- aplicar age_delta_years;
- modificar ropa;
- actualizar campo de modelo;
- agregar modelo;
- remover modelo;
- actualizar marca;
- regenerar runtime;
- regenerar prompts;
- regenerar agentes;
- actualizar proyecto por nueva versión de motor;
- actualizar proyecto manteniendo misma versión de motor.

Cada update debe:

- identificar superficies afectadas;
- preservar superficies no afectadas;
- actualizar manifests;
- actualizar runtime impactado;
- actualizar evidence pointers;
- actualizar companion;
- actualizar no-drift ledger;
- reabrir ZIP final;
- validar recomputacionalmente.

No debe:

- regenerar todo si no es necesario;
- mezclar modelos;
- contaminar modelos no involucrados;
- perder canon;
- borrar evidencia útil;
- dejar placeholders;
- declarar PASS sin revalidar.

11. Política de actualización de motor

El motor debe poder actualizarse con precisión quirúrgica.

Toda actualización de motor debe:

1. validar base;
2. leer reglas vigentes;
3. identificar cambio exacto;
4. modificar solo superficies necesarias;
5. eliminar obsoletos reemplazados;
6. no duplicar lógica;
7. no crear factory paralelo;
8. no crear validator paralelo;
9. no acumular historial activo;

10. correr matriz máxima;
 11. generar release surfaces;
 12. entregar archivos exactos;
 13. bloquear si algo no pasa.
-

12. Política de auditoría máxima

No se debe hablar de matriz mínima como estándar de cierre.

Para cierre de motor se exige matriz máxima.

12.1 Auditorías máximas de motor

El motor debe pasar auditorías de:

- integridad ZIP;
- SHA companion externo;
- compresión;
- testzip;
- no stored;
- estructura;
- numeración de carpetas;
- no-bloat;
- no-history;
- no-staging;
- no-temp;
- no-logs;
- no-duplicados no gobernados;
- no-leakage de proyecto;
- no modelos demo activos;
- no defaults de marca;
- no sufijos semánticos;
- factory único;
- validator único;
- schemas conectados;
- Profile360 preservado;
- TechExt preservado;
- evidence lineage preservado;
- runtime contracts;
- agent contracts;
- prompt pack A-J;
- update contracts;
- migration contracts;
- reference image;
- safe rewrite;
- adult editorial policy;
- brand registry;
- project output contract;

- CLI lifecycle;
- generate;
- validate;
- update;
- migrate;
- mutation/self-test;
- N1..N10 x niveles de input.

12.2 Matriz máxima de generación

Para motor release:

```
N1..N10 modelos
x
low-info
intermediate-info
full-info
=
30 generaciones
```

Cada generación debe pasar:

- generate rc=0;
- ZIP final creado;
- companion SHA;
- testzip;
- stored=0;
- validate recomputado;
- duplicate allowlist;
- runtime 10+N;
- human-readable canon;
- prompt packs A-J;
- no sentinels;
- no placeholders;
- no staging;
- no temp;
- no heartbeat;
- no proceso colgado.

12.3 Matriz máxima de updates

Probar:

- age_delta_years;
- age_evolution;
- set_wardrobe;
- update_model_field;
- brand_update;
- runtime_regeneration;

- add_model;
- remove_model.

En formatos:

- top-level alias;
- operations[];
- operación singular si se soporta.

12.4 Matriz máxima de migraciones

Probar:

- proyecto versión actual → misma versión;
- proyecto versión anterior compatible → versión nueva;
- update-project-by-engine;
- migrate-project;
- proyecto con 1 modelo;
- proyecto con 2 modelos;
- proyecto con 10 modelos;
- proyecto con poca información;
- proyecto con información completa.

12.5 Matriz de agentes

Probar:

- ChatGPT load;
- Copilot load;
- lectura 10+N;
- selector de modelo;
- escena individual;
- escena grupal;
- imagen desde texto;
- prompt desde texto;
- prompt desde referencia;
- imagen desde referencia;
- video;
- voz;
- música/Suno;
- texto/persona;
- safe rewrite;
- QA;
- brand/logo;
- bloqueo correcto.

13. Regla de 100% / 10-10

El motor o proyecto solo se entrega si todas las pruebas obligatorias pasan.

Regla:

Si una prueba falla, no se entrega.
Si una simulación falla, no se entrega.
Si una auditoría falla, no se entrega.
Si un comando no retorna limpio, no se entrega.
Si un PASS no es recomputado, no se entrega.

El proceso debe iterar:

corregir → auditar → simular → validar → repetir

Hasta obtener:

VALIDATORS_FAIL=0
BLOCKING_WARNINGS=0
FAIL_CODES=[]
SCORE=10/10

No se acepta:

- "suficiente";
- "casi";
- "parcial";
- "manual workaround";
- "PASS declarado";
- "timeout pero escribió output";
- "clean fail no documentado";
- "pendiente posterior" en cierre de motor, salvo fases posteriores explícitas como Proyecto Demo o carga real de agente.

14. Política de salida externa del motor

La salida externa debe definirse por prompt.

Cuando se exige entrega 7/7, debe ser exactamente:

1. IDUNEX_MOTOR_vX.Y.Z.zip
2. IDUNEX_MOTOR_vX.Y.Z.zip.sha256
3. IDUNEX_MOTOR_vX.Y.Z_RELEASE_CERTIFICATE.txt
4. IDUNEX_MOTOR_vX.Y.Z_MANUAL_TECNICO_ALCANCE_LATINOSFLOW.pdf
5. IDUNEX_MOTOR_vX.Y.Z_MANUAL_DE_TRABAJO_LATINOSFLOW.pdf

6. IDUNEX_PROMPT_CANONICO_PROJECT_000_DEMO_AUTORIDAD_MOTOR.txt
7. IDUNEX_PROMPT_SUITE_PLANTILLAS_PROYECTOS_AUTORIDAD_MOTOR_.docx

Si en otra versión futura se define otra salida, debe estar explícita por prompt y contrato.

No entregar archivos extra.

15. Estructura oficial sugerida del motor

```
IDUNEX/  
00_INDEX/  
01_CANON_REGISTRIES/  
02_RESEARCH_CORPUS/  
03_PROJECT_FACTORY/  
04_AGENT_FACTORY/  
05_RUNTIME_CORE_LIBRARY/  
06_MULTIMODAL_CONTRACTS/  
07_VALIDATION_QA_GAUNTLET/  
08_EVIDENCE_LINEAGE/  
09_TEMPLATES_FIXTURES/  
10_INTERNAL_MANUALS/  
11_RELEASE_INTERNAL/  
12_OUTPUT_CONTRACTS/  
13_UPDATE_MIGRATION/  
14_HISTORICAL_NON_AUTHORITY/  
99_MANIFESTS_SHA_LINEAGE/
```

15.1 00_INDEX

Mapa maestro, autoridad, rutas, módulos, lectura interna.

15.2 01_CANON_REGISTRIES

Profile360, TechExt, schemas de identidad, campos canónicos.

15.3 02_RESEARCH_CORPUS

Investigaciones normalizadas por dominio:

- visual;
- físico;
- moda;
- ambientes;
- cámara;
- iluminación;
- voz;

- ElevenLabs;
- música;
- Suno;
- texto;
- personalidad;
- marca;
- seguridad;
- agentes;
- QA.

Debe estar deduplicado y conectado.

15.4 03_PROJECT_FACTORY

Generación, validación, finalizer, output contract, duplicate governance, project creation.

15.5 04_AGENT_FACTORY

Runtime ChatGPT/Copilot, load maps, task registry, read map, selector, reference handler.

15.6 05_RUNTIME_CORE_LIBRARY

10 base templates, full runtime model template, prompt-ready blocks, fallback fixes.

15.7 06_MULTIMODAL_CONTRACTS

Imagen, video, voz, audio, Suno, ElevenLabs, texto, referencia, QA, vendor handoff.

15.8 07_VALIDATION_QA_GAUNTLET

Auditorías, validators, mutation tests, stress tests, matrix tests.

15.9 08_EVIDENCE_LINEAGE

Lineage, source pointers, coverage, compact evidence.

15.10 09_TEMPLATES_FIXTURES

Blueprints y fixtures mínimos, no fixtures pesados repetidos.

15.11 10_INTERNAL_MANUALS

Manual técnico, manual de trabajo, guías internas.

15.12 11_RELEASE_INTERNAL

Release surfaces internas no autorreferenciales.

15.13 12_OUTPUT_CONTRACTS

Contratos de salida de motor, proyecto, agente y Demo.

15.14 13_UPDATE_MIGRATION

Reglas de actualización de motor, proyecto y migraciones.

15.15 14_HISTORICAL_NON_AUTHORITY

Solo evidencia compacta no autoritativa, si es estrictamente necesaria.

15.16 99_MANIFESTS_SHA_LINEAGE

Manifests internos, hashes de archivos internos, lineage no autorreferencial.

16. Política ZIP y autoridad externa

El SHA integral del ZIP completo y sus bytes absolutos deben tener autoridad externa.

No deben autocertificarse dentro del propio ZIP.

Regla estable:

ZIP-EXT-001: Whole ZIP external authority

Autoridad externa:

.zip.sha256
release certificate externo
companion externo

Validable internamente:

- file count;
- entry count;
- directories;
- stored count;
- compression;
- manifests;
- duplicate groups;
- allowlist;
- CRC/testzip;
- runtime contracts;

- prompt packs;
- no sentinels.

17. Runtime 10+N

Cada proyecto debe generar runtime para:

- ChatGPT;
- Copilot.

Cada runtime:

10 archivos base + 1 full runtime canon por modelo

El archivo por modelo debe ser completo, no resumen.

Debe incluir:

- visual;
- físico;
- rostro;
- piel;
- ojos;
- cabello;
- cuerpo;
- ropa;
- voz;
- personalidad;
- comportamiento;
- movimiento;
- música;
- texto;
- prompts;
- negativos;
- fallback fixes;
- QC;
- evidence pointers.

18. Prompt packs A-J

Todo prompt pack debe incluir:

- A) HEADER
B) SCENE
C) COMPOSITION

D) LIGHTING
E) WARDROBE / PROPS
F) CAMERA / TECH
G) NEGATIVE / AVOID
H) PARAMS
I) QC CHECKLIST
J) FALLBACK FIXES

Aplica a:

- imagen;
- video;
- voz;
- música/Suno;
- ElevenLabs;
- texto/persona;
- referencia visual;
- handoff proveedor;
- QA;
- certificación.

19. Política de marca

El motor no tiene marca fija.

Cada proyecto define su marca.

Debe existir:

PROJECT_BRAND_REGISTRY

Si no hay logo verificado:

LOGO_ASSET_NOT_VERIFIED

No inventar logo exacto.

20. Política de referencia visual

Permitido copiar desde referencia:

- pose;
- ropa;
- composición;

- cámara;
- iluminación;
- energía;
- estilo permitido.

Prohibido copiar:

- identidad facial real;
 - cuerpo identificable real;
 - edad real;
 - marca/logo no autorizado;
 - persona real no autorizada.
-

21. Política adulto editorial segura

Permitido para modelos ficticios adultos:

- ropa de baño;
- lencería;
- glamour;
- pose sensual;
- editorial fashion;
- performance wardrobe cubierto.

Bloqueado:

- desnudez;
 - sexo explícito;
 - pornografía;
 - exposición íntima;
 - apariencia menor;
 - school-coded sexualizado;
 - coerción;
 - evasión de políticas.
-

22. Política de no-bloat / no-history

No guardar:

- logs largos;
- auditorías humanas completas;
- staging;
- temp;
- heartbeat;
- ZIPs intermedios;
- proyectos generados;
- versiones viejas activas;

- backups;
- deprecated activo;
- outputs de prueba;
- historial acumulado.

Solo permitir:

- changelog compacto;
- evidence compacta;
- lineage útil;
- referencias no autoritativas claramente marcadas.

23. Sentinels bloqueantes

Bloquear si aparece en superficies activas:

```
FINAL_ZIP_REOPEN_REQUIRED  
RECOMPUTED_ON_FINAL_REOPEN  
SOURCE_PATH_NOT_AVAILABLE  
NOT_APPLICABLE_WITH_REASON sin razón válida  
UNKNOWN_TARGET_ENGINE_BLOCKED para versión compatible  
PARTIAL_ZIP  
HEARTBEAT  
TEMP_NON_DELIVERY  
STAGING  
SIMULATION_ONLY_PASS  
DECLARED_PASS_ONLY
```

24. Certificación creativa

El motor no certifica outputs creativos.

Siempre:

```
CREATIVE_OUTPUT_CERTIFIED=FALSE
```

Solo puede cambiar en un proyecto/output si existe:

- asset real;
- sidecar;
- hash;
- reviewer;
- lineage;
- QA expected;

- QA actual;
- EXECUTED_PASS individual.

25. Cierre ideal de motor

El motor queda apto para pasar a Proyecto Demo cuando cumple:

```
SEMANTIC_VERSION=x.x.x
OUTPUT_EXTERNAL_CONTRACT=PASS
FINAL_ZIP_TESTZIP=PASS
NO_ZIP_STORED_INTERNAL_FILES=PASS
COMPANION_SHA_MATCH=PASS
VALIDATORS_FAIL=0
BLOCKING_WARNINGS=0
FAIL_CODES=[]
NO_ACTIVE_BLOAT=PASS
NO_ACTIVE_PROJECT_LEAKAGE=PASS
PROFILE360_REGISTRY_COVERAGE=PASS
TECHEXT_FIELD_COVERAGE=PASS
RESEARCH_CORPUS_CONNECTEDNESS=PASS
RUNTIME_CHATGPT_COPILOT_10_PLUS_N_CONTRACT=PASS
PROMPT_PACK_A_J_CONTRACT=PASS
N1_TO_N10_X3_MATRIX=PASS
UPDATE_MATRIX=PASS
MIGRATION_MATRIX=PASS
MUTATION_SELF_TEST=PASS
CLI_LIFECYCLE=PASS
CREATIVE_OUTPUT_CERTIFIED=FALSE
```

26. Regla final de operación

El motor debe ser el corazón del sistema IDUNEX.

Debe gobernar:

- qué se crea;
- cómo se crea;
- con qué investigación se respalda;
- cómo se valida;
- cómo se actualiza;
- cómo se migra;
- cómo se carga en agentes;
- cómo se evita drift;
- cómo se evita bloat;
- cómo se bloquea un falso PASS.

Regla final:

```
Si es motor, gobierna.  
Si es proyecto, materializa.  
Si es agente, ejecuta.  
Si es investigación, respalda.  
Si es auditoría, recompute.  
Si es histórico, no manda.  
Si no pasa 100%, no se entrega.
```

27. Contrato de entrega final de proyecto

Todo proyecto IDUNEX generado por el motor debe tener una entrega final explícita, cerrada, auditable y recomputable.

La salida exacta puede variar según el contrato definido por el motor o por el prompt de creación, pero como estándar base todo proyecto debe entregar como mínimo:

1. IDUNEX_PROJECT_<PROJECT_CODE>_<SEMANTIC_VERSION>.zip
2. IDUNEX_PROJECT_<PROJECT_CODE>_<SEMANTIC_VERSION>.zip.sha256
3. IDUNEX_PROJECT_<PROJECT_CODE>_<SEMANTIC_VERSION>_RELEASE_CERTIFICATE.txt
4. IDUNEX_PROJECT_<PROJECT_CODE>_<SEMANTIC_VERSION>_FINAL_AUDIT_REPORT.md
5. IDUNEX_PROJECT_<PROJECT_CODE>_<SEMANTIC_VERSION>_README_FOR_HUMAN_OPERATOR.md

El ZIP del proyecto debe contener internamente, como mínimo:

```
PROJECT_MANIFEST  
PROJECT_BRAND_REGISTRY  
MODEL_REGISTRY  
FULL_PROFILE360 por modelo  
TECHTEXT por modelo  
HUMAN_READABLE_VISUAL_CANON por modelo  
VOICE_CANON por modelo si aplica  
WARDROBE_CANON por modelo  
ENVIRONMENT_CANON  
TEXT_PERSONA_CANON  
MUSIC_PERSONA_CANON / SUNO_READY si aplica  
RUNTIME_CHATGPT 10+N  
RUNTIME_COPILOT 10+N  
README_FOR_HUMAN_OPERATOR  
TEST_PROMPTS_BY_MODALITY  
STATIC_SMOKE_TEST_MATRIX  
PROMPT_PACKS_A_J  
EVIDENCE_LINEAGE
```

```
FIELD_SOURCE_TRACE_LEDGERS
NO_DRIFT_LEDGERS
DUPLICATE_ALLOWLIST
MANIFESTS_SHA
FINAL_MACHINE_AUDIT_SUMMARY
PROJECT_RELEASE_CERTIFICATE interno
AGENT_LOAD_PROTOCOL
AGENT_LOAD_AUDIT_CHECKLIST
```

Regla:

Un proyecto no queda entregado porque exista un ZIP.
Un proyecto queda entregado solo cuando ZIP + SHA + certificado + auditoría + runtime 10+N están sincronizados y validados.

28. Contrato de carpetas oficiales del proyecto

Todo proyecto debe organizarse con estructura clara, numerada y sin mezcla con el motor.

Estructura sugerida:

```
IDUNEX_PROJECT_<PROJECT_CODE>/
00_PROJECT_INDEX/
01_PROJECT_AUTHORITY/
02_BRAND_REGISTRY/
03_MODELS/
04_CANON_360/
05_MULTIMODAL_OUTPUTS_CONTRACTS/
06_AGENTS/
07_RUNTIME_UPLOAD/
08_EVIDENCE_LINEAGE/
09_QA_VALIDATORS/
10_PROMPT_PACKS/
11_TEST_PROMPTS/
12_SMOKE_TEST_MATRIX/
13_UPDATE_MIGRATION/
14_RELEASE/
99_MANIFESTS_SHA/
```

28.1 00_PROJECT_INDEX

Debe contener:

- índice del proyecto;
- mapa de carpetas;

- lectura rápida;
- resumen de modelos;
- resumen de marca;
- rutas runtime;
- estado de auditoría.

28.2 01_PROJECT_AUTHORITY

Debe contener:

- prompt de creación usado;
- input autorizado del proyecto;
- restricciones;
- owner;
- uso previsto;
- alcance legal/creativo;
- decisiones del motor.

No debe contener prompts viejos, handoffs obsoletos ni instrucciones contradictorias.

28.3 02_BRAND_REGISTRY

Debe contener:

- PROJECT_BRAND_REGISTRY;
- logo verificado o LOGO_ASSET_NOT_VERIFIED ;
- paleta;
- reglas de uso;
- derechos;
- restricciones de marca;
- fallback visual.

28.4 03_MODELS

Debe contener:

- MODEL_REGISTRY;
- lista de modelos;
- identificadores;
- edad ficticia adulta;
- rol;
- modalidad activa;
- bloqueo de identidad;
- relaciones entre modelos si aplica.

28.5 04_CANON_360

Debe contener por modelo:

- Profile360;

- TechExt;
- human-readable visual canon;
- rostro;
- piel;
- ojos;
- cabello;
- cuerpo;
- voz;
- personalidad;
- conducta;
- guardarropa;
- ambientes;
- movimiento;
- música/persona;
- texto/persona;
- negativos;
- fallback fixes;
- QA checklist.

28.6 05_MULTIMODAL_OUTPUTS_CONTRACTS

Debe contener contratos para:

- imagen;
- video;
- voz;
- audio;
- música/Suno;
- ElevenLabs;
- texto/persona;
- referencia visual;
- QA;
- certificación.

28.7 06_AGENTS

Debe contener:

- configuración ChatGPT;
- configuración Copilot;
- task registry;
- read map;
- agent rules;
- vendor limitations;
- safe rewrite;
- reference handler.

28.8 07_RUNTIME_UPLOAD

Debe contener exactamente los paquetes que se cargan a agente:

CHATGPT/10_BASE_PLUS_N_MODELS
COPILOT/10_BASE_PLUS_N_MODELS

No debe contener archivos extra que confundan al agente.

28.9 08_EVIDENCE_LINEAGE

Debe contener:

- source pointers;
- field source trace;
- evidence maps;
- research-derived canon;
- companion ledgers.

28.10 09_QA_VALIDATORS

Debe contener validadores de proyecto y resultados recomputados.

28.11 10_PROMPT_PACKS

Debe contener prompt packs A-J por modalidad.

28.12 11_TEST_PROMPTS

Debe contener prompts mínimos, intermedios y full-info para probar el proyecto.

28.13 12_SMOKE_TEST_MATRIX

Debe contener matriz de pruebas del proyecto.

28.14 13_UPDATE_MIGRATION

Debe contener reglas para actualizar este proyecto sin drift.

28.15 14_RELEASE

Debe contener certificado, resumen final, reporte final y cierre.

28.16 99_MANIFESTS_SHA

Debe contener hashes internos, manifests y companion references no autorreferenciales.

29. Contrato de auditoría de proyecto

Cada proyecto debe pasar auditoría propia antes de considerarse apto.

La auditoría de proyecto no repite necesariamente toda la matriz máxima del motor, pero sí debe recomputar todo lo específico del proyecto.

Debe validar:

```
ZIP final reabierto
external SHA companion
testzip PASS
stored=0
entry_count/file_count/directories
duplicate allowlist real
PROJECT_BRAND_REGISTRY
MODEL_REGISTRY
Profile360 por modelo
TechExt por modelo
human-readable canon por modelo
runtime ChatGPT 10+N
runtime Copilot 10+N
prompt packs A-J
README_FOR_HUMAN_OPERATOR
TEST_PROMPTS_BY_MODALITY
STATIC_SMOKE_TEST_MATRIX
companion ledgers reales
no-drift ledger
release certificate
final machine audit summary
no placeholders
no sentinels
no staging/temp/heartbeat
CREATIVE_OUTPUT_CERTIFIED=FALSE
```

Bloquear si:

```
VALIDATORS_FAIL > 0
BLOCKING_WARNINGS > 0
FAIL_CODES != []
PASS declarado sin recomputar
runtime incompleto
modelo con canon incompleto
brand registry ausente
logo inventado
companion ledger sin source real
```

duplicado no gobernado
proceso no retorna limpio

30. Contrato de carga de agente por proyecto

Un proyecto no queda listo para producción solo por tener ZIP. Debe estar listo para carga operativa en agentes.

Cada proyecto debe generar un paquete claro para:

ChatGPT
Copilot

Cada paquete debe contener:

10 archivos base + 1 full runtime canon por modelo

Además debe existir:

README_FOR_HUMAN_OPERATOR
AGENT_LOAD_PROTOCOL
AGENT_LOAD_AUDIT_CHECKLIST
TASK_REGISTRY
READ_MAP
MODEL_SELECTOR_RULES
REFERENCE_IMAGE_HANDLER
SAFE_REWRITE_POLICY
NEGATIVE_AVOID_GLOBAL
FALLBACK_FIXES_GLOBAL

El operador humano debe poder responder rápidamente:

Qué archivos cargar
En qué orden cargar
Qué modelos están disponibles
Qué tareas soporta el agente
Qué no debe hacer el agente
Cómo probar que cargó bien
Cómo reportar vendor limitation

31. Estado final de proyecto

Todo proyecto debe tener un estado explícito.

Estados permitidos:

```
PROJECT_GENERATED_NOT_AUDITED
PROJECT_AUDIT_FAIL
PROJECT_AUDIT_PASS
PROJECT_AGENT_LOAD_PENDING
PROJECT_AGENT_LOAD_FAIL
PROJECT_AGENT_LOAD_PASS
PROJECT_READY_FOR_CONTROLLED_USE
PROJECT_READY_FOR_PRODUCTION
PROJECT_UPDATE_REQUIRED
PROJECT_DEPRECATED_NON_AUTHORITY
```

Un proyecto solo puede llegar a `PROJECT_READY_FOR_PRODUCTION` si pasa:

1. auditoría recomputacional del proyecto
2. runtime ChatGPT 10+N validado
3. runtime Copilot 10+N validado o vendor limitation documentada
4. smoke tests propios
5. no-drift checks
6. brand/assets checks
7. truthfulness checks

Regla:

Motor productivo no significa proyecto automáticamente productivo.
Motor productivo significa que el factory está certificado.
Proyecto productivo significa que ese proyecto concreto pasó su propia auditoría.

Pero para evitar redundancia:

Un proyecto normal no repite la matriz máxima del motor si el motor ya pasó Proyecto Demo.
Un proyecto normal ejecuta auditoría recomputacional específica.
Un proyecto crítico ejecuta auditoría extendida.
Un update ejecuta auditoría delta + no-drift.
Una migración ejecuta auditoría de migración + revalidación.

32. Regla final ampliada

El sistema IDUNEX debe quedar gobernado por tres contratos mayores:

```
ENGINE_OUTPUT_CONTRACT
PROJECT_OUTPUT_CONTRACT
AGENT_LOAD_CONTRACT
```

El motor gobierna.

El proyecto materializa.

El agente ejecuta.

Nada debe quedar implícito.

Regla final:

```
Si se crea motor, debe cerrar como motor.
Si se crea proyecto, debe cerrar como proyecto.
Si se carga agente, debe cerrar como carga real.
Si una capa falla, no se arrastra PASS desde otra capa.
Si una capa pasa, habilita la siguiente capa, pero no la certifica
automáticamente.
```

El cierre correcto siempre debe respetar la secuencia:

```
MOTOR → PROYECTO DEMO → AUDITORÍA DEMO → CARGA AGENTE → AUDITORÍA AGENTE →
PRODUCTIVO
```

Y para proyectos futuros:

```
MOTOR PRODUCTIVO → PROYECTO NUEVO → AUDITORÍA PROYECTO → CARGA AGENTE SI
APLICA → USO CONTROLADO / PRODUCCIÓN
```